

Índice

Neovim desde cero	1
Cómo está armado (y qué suelen quejarse otros cursos)	1
0. Manos en el teclado	2
1. Instalar esta config	2
2. Modos (el único truco que hay que pillar)	3
3. Moverse sin flechas	3
4. La gramática: operador + movimiento	4
5. Objetos de texto (antes de cualquier plugin)	5
6. Buscar y sustituir	5
7. Buffers, ventanas, pestañas	5
8. Registros y macros	6
9. Ayuda, guardar, salir	6
10. Esta config como IDE	7
11. Plugins de este repo (cuándo, no el catálogo entero)	7
12. IdeaVim	8
13. Cuando se rompe	8
Orden sugerido en clase	8

Neovim desde cero

Este texto es el cuaderno del curso. La idea no es memorizar 200 atajos el primer día. Es hablar el idioma de Vim y, cuando eso ya no duela, usar esta configuración como IDE.

Espacio es el *leader*. Si lees `<leader>ff`, pulsa Espacio y luego `ff`. Si pulsas solo Espacio y esperas, which-key te enseña el menú. Espacio dos veces quita el resaltado de una búsqueda; no abre el buscador de archivos.

Cuando dudes de un atajo de *esta* config: `:Telescope keymaps`.

`:help` es el manual de verdad. Este cuaderno no lo sustituye.

Cómo está armado (y qué suelen quejarse otros cursos)

El tutor oficial (`vimtutor`), el manual de Bram (`:help usr_02`) y guías como *Learn Vim the Smart Way* y *vim-galore* coinciden en lo mismo:

1. Primero modos, movimiento y la gramática **operador + movimiento**.
2. Luego objetos de texto (`diw`, `ci`), no al revés.
3. Buffers, ventanas y saltos *antes* de Telescope.
4. Registros, macros y `:s` antes de “el plugin que lo hace por ti”.
5. Plugins al final. vim-galore lo dice claro: aprende Vim bien *antes* de llenarlo de extensiones.

Lo que la gente suele echar en falta cuando un curso “de Neovim” empieza por lazy.nvim: no saber salir, no entender `d` frente a `x`, no haber usado nunca `ciw`, no saber qué es un buffer. Aquí eso va primero. Los plugins de este repo van después, y solo con teclas que existen en el Lua.

IntelliJ con IdeaVim y Vim clásico no se “validan” desde esta guía: hay que abrir el IDE. Neovim sí: `nvim --headless "+lua print('boot-ok')"+qa`.

0. Manos en el teclado

Vim premia no mirar el teclado. `h j k l` están en la fila home a propósito.

Practica 15 minutos al día, sin este editor:

- keybr.com — te va soltando letras.
- monkeytype.com — velocidad.
- typingclub.com — lecciones (hay español).

Cuando ya no busques la `J` con los ojos, abre openvim.com (tutorial en el navegador) o, mejor, el tutor que trae Vim:

```
nvim +Tutor
```

(En Vim clásico el comando es `vimtutor`. En Neovim: `nvim +Tutor`.)

Haz el tutor entero una vez. Aburre. Funciona.

Entrenador de escritorio (Windows / Linux / macOS, sin red): carpeta `trainer/`. `flutter test` y `flutter run`. No es Neovim; solo el subconjunto documentado en Acerca de. Builds de GitHub **sin firma**.

1. Instalar esta config

Neovim **0.12 o más**. No clones el repo *dentro* de `~/.config/nvim`.

Windows

```
winget install Neovim.Neovim Git.Git OpenJS.NodeJS DEVCOM.JetBrainsMonoNerdFont
npm install -g tree-sitter-cli
git clone git@github.com:chochy2001/Neovim-Vim_Configuration.git $HOME\Neovim-Vim_Configuration
New-Item -ItemType Junction -Path "$env:LOCALAPPDATA\nvim" -Target "$HOME\Neovim-Vim_Configuration\.\config"
Copy-Item "$HOME\Neovim-Vim_Configuration\.\ideavimrc" "$HOME\.\ideavimrc"
```

En el terminal pon fuente **JetBrainsMono NFM** y ciérralo. Sin esa fuente los iconos salen como recuadros.

macOS / Linux

```
# neovim, git, node, compilador C, ripgrep, fd - con brew o el paquete de la distro
npm install -g tree-sitter-cli
git clone git@github.com:chochy2001/Neovim-Vim_Configuration.git ~/Neovim-Vim_Configuration
mkdir -p ~/.config
ln -sfn ~/Neovim-Vim_Configuration/.config/nvim ~/.config/nvim
ln -sf ~/Neovim-Vim_Configuration/.ideavimrc ~/.ideavimrc
```

Primera vez: `nvim` y espera a que lazy.nvim instale. `:Lazy` para mirar. Mason instala servidores de lenguaje en segundo plano.

Comprobar arranque:

```
nvim --headless "+lua print('boot-ok')"+qa
```

Tiene que salir `boot-ok`, sin `Error detected`.

2. Modos (el único truco que hay que pillar)

Vim no es un bloc de notas con atajos. Es un editor **modal**: la misma tecla hace cosas distintas según el modo.

Estás en...	Entras con	Salas con	Sirve para
Normal	al abrir, o Esc	—	moverte y dar órdenes
Insertar	i a o I A o O	Esc o jj	escribir
Visual	v V Ctrl-v	Esc	seleccionar
Línea de comandos	: / ?	Enter o Esc	guardar, buscar, ayuda

Si te pierdes: **Esc Esc**. El pitido (o nada) significa “ya estás en Normal”.

En esta config, **jj** también sale de insertar. **Esc** sigue existiendo. Aprende los dos.

Ejercicio. `nvim prueba.txt` → **i** → escribe tu nombre → **Esc** → **i** otra vez → **jj**. Mira la esquina: la statusline dice el modo (esta config oculta `-- INSERT --` nativo; `lualine` lo muestra).

3. Moverse sin flechas

Las flechas funcionan. Te hacen sacar la mano de las letras. El manual de Vim insiste en **h j k l** por eso.

```
  k
h  l
  j
```

h está a la izquierda, **l** a la derecha, **j** apunta abajo.

Tecla	Qué hace
h j k l	un carácter / una línea
w b e	palabra siguiente, anterior, fin de palabra
W B E	igual, pero saltando puntuación
O ^ \$	inicio de línea, primer no-espacio, final
gg G	primera línea, última
{ }	párrafo
()	frase
fx tx	en esta línea: hasta la x / antes de la x
Fx Tx	lo mismo hacia atrás
; ,	repetir f/t
%	el paréntesis, corchete o llave de enfrente
H M L	alto, medio, bajo de la pantalla
Ctrl-d Ctrl-u	media página
Ctrl-f Ctrl-b	página

Delante de casi todo puedes poner un **número**: **5j** baja cinco líneas, **3w** tres palabras, **10G** va a la línea 10.

Ejercicio. Abre cualquier texto largo. Ve al final con **G**, al principio con **gg**, a la línea 20 con **20G**. Recorre un párrafo con **}** y vuelve con **{**. Busca una coma en la línea con **f,**.

4. La gramática: operador + movimiento

Esto es el núcleo. Bram y *Learn Vim* lo llaman gramática. Un **operador** espera un **movimiento** (o un objeto de texto).

Operadores que vas a usar todos los días:

Operador	Significado
d	borrar (<i>delete</i>)
c	borrar y entrar a insertar (<i>change</i>)
y	copiar (<i>yank</i>)
> <	indentar / desindentar
=	reindentar (con LSP/treesitter más adelante)
gU gu g~	mayúsculas, minúsculas, invertir

Movimiento = *dónde*. Entonces:

- **dw** borrar hasta el inicio de la siguiente palabra
- **d\$** borrar hasta el final de la línea
- **dG** borrar hasta el final del archivo
- **c3w** cambiar tres palabras
- **yip** copiar el párrafo interior (esto ya es objeto de texto; el siguiente capítulo)

Atajos que *parecen* comandos sueltos pero son esta gramática con un movimiento especial **_** (la línea actual): **dd**, **cc**, **yy**. El tutor lo enseña como “borrar línea”; por debajo es **d** sobre la línea.

Otros que conviene tener en los dedos:

Tecla	Qué hace
x	borrar un carácter (dl)
X	borrar el de atrás (dh)
s	cambiar un carácter — ojo : en <i>esta</i> config, s es Flash (saltar). Para “change char” usa cl
r + letra	reemplazar un carácter sin entrar a insertar
J	juntar esta línea con la de abajo
p P	pegar después / antes
.	repetir el último cambio
u	deshacer
Ctrl-r	rehacer

Insertar con intención:

Tecla	Dónde inserta
i	antes del cursor
a	después del cursor
I	al primer no-espacio de la línea
A	al final de la línea
o	línea nueva debajo, ya en insertar
O	línea nueva encima

Ejercicio. Escribe tres líneas. **dd** la del medio. **p** la pega debajo. **u** la devuelve. **3j** no hace falta: estás en un archivo corto. Cambia una palabra con **cw** (escribe otra, **Esc**). Pulsa **.** en otra palabra: Vim repite el cambio.

5. Objetos de texto (antes de cualquier plugin)

Un movimiento va en *una* dirección. Un **objeto** es una cosa alrededor del cursor: una palabra, una frase, lo que hay entre comillas.

Empiezan por *i* (*inner*, por dentro) o *a* (*around*, con el envoltorio).

Objeto	Ejemplo	Efecto
iw aw	diw	borra la palabra, con o sin espacio pegado
is as	cis	cambia la frase
ip ap	yip	copia el párrafo
i" a"	ci"	cambia lo que hay entre comillas
i' a'	da'	borra comillas simples y contenido
i(a(	ci(	cambia dentro de paréntesis (también ib)
i{ a{	da{	borra el bloque { ... }
i[a[	vi[	selecciona dentro de corchetes
it at	cit	dentro de un tag HTML/XML
i< a<	ci<	dentro de <>

La receta que más se usa al programar: **ciw** (cambiar esta palabra), **ci"** (cambiar el string), **ci{** (cambiar el bloque).

Visual: **v** carácter, **V** líneas, **Ctrl-v** bloque (columnas). Con la selección hecha, **d c y >** actúan sobre ella. **o** salta al otro extremo de la selección.

En esta config, **>** y **<** en visual **mantienen** la selección (para indentar varias veces).

Ejercicio. Escribe `foo("hola mundo")`. Cursor en `hola`. **ci"** → escribe `adiós` → **Esc**. Cursor en `foo`. **ciw** → `bar`. **va(** y mira qué se selecciona.

6. Buscar y sustituir

/palabra Enter busca hacia adelante. **?palabra** hacia atrás. **n** siguiente, **N** anterior. En esta config, **n/N** además **centran** la pantalla.

***** busca la palabra bajo el cursor hacia adelante, **#** hacia atrás.

:nohl quita el resaltado. Aquí: `<leader><leader>`.

Sustituir (línea de comandos):

```
:s/viejo/nuevo/      « solo la primera vez en esta línea
:s/viejo/nuevo/g     « todas en esta línea
:%s/viejo/nuevo/g    « todo el archivo
:%s/viejo/nuevo/gc   « todo el archivo, preguntando
```

% es el rango "archivo entero". **:'<,'>s/** (sale solo si venías de visual) actúa sobre la selección.

Ejercicio. Pon tres veces la palabra `gato`. **:%s/gato/perro/g** y comprueba.

7. Buffers, ventanas, pestañas

Olvida un momento Telescope. En Vim:

- **Buffer** = un archivo cargado en memoria. Puede no verse.

- **Ventana** = un hueco que *mira* a un buffer.
- **Pestaña** = un conjunto de ventanas.

:e archivo abre (o crea) un buffer. :ls lista. :b siguiente o :b 2 cambia. :bd cierra el buffer.

Ventanas nativas:

Tecla	Qué hace
Ctrl-w s	partir horizontal
Ctrl-w v	partir vertical
Ctrl-w h/j/k/l	ir a esa ventana
Ctrl-w c	cerrar ventana
Ctrl-w o	dejar solo esta

Esta config añade (mismo significado, con leader): <leader>sv vertical, sh horizontal, sc cerrar, so solo esta, wh wj wk wl moverse.

Buffers con pestañas de bufferline: Shift-l / Shift-h siguiente/anterior, <leader>bn bp bd.

Salto. Cada vez que haces un salto grande (G, /, gd más adelante), Vim guarda el sitio. Ctrl-o vuelve atrás, Ctrl-i adelante. :jumps lista. <leader>b en esta config es “atrás” de historial (como el Back del IDE); espera 300 ms porque bn/bp empiezan igual.

Marcas. ma guarda la posición en la marca a. 'a vuelve al inicio de esa línea, `a a la columna exacta. Marcas A-Z son globales (otro archivo).

Ejercicio. Abre dos archivos con :e. Parte la ventana Ctrl-w v. Salta con G, vuelve con Ctrl-o.

8. Registros y macros

Yank (y) no es “el portapapeles de Windows” nada más. Hay cajones:

- " sin nombre: último yank o borrado
- 0 último yank
- 1–9 últimos borrados
- a–z los tuyos ("ayy copia la línea al registro a)
- + portapapeles del sistema (esta config usa unnamedplus)
- _ agujero negro ("dd borra sin ensuciar el yank)

:reg enseña el contenido.

Macros: q + letra para grabar, q para parar, @a para reproducir, @@ la última. Ejemplo: en una lista, qa → I-Esc j q y luego 10@a.

Ejercicio. Copia una línea con yy. Muévete. p. Ahora "ayy en otra línea y "ap.

9. Ayuda, guardar, salir

Comando	Qué hace
:w	guardar
:q	salir si no hay cambios
:wq o ZZ	guardar y salir
:q!	salir tirando los cambios
:e!	recargar el archivo del disco

Comando	Qué hace
<code>:help</code>	manual
<code>:help x</code>	la tecla <code>x</code>
<code>:help :w</code>	el comando <code>:w</code>
<code>:help 'number'</code>	una opción
<code>Ctrl-]</code>	seguir un enlace en el help
<code>Ctrl-t</code> o <code>Ctrl-o</code>	volver

En el help, `Ctrl-d` después de un tema incompleto lista coincidencias.

10. Esta config como IDE

Hasta aquí todo es Vim. Lo siguiente *añade* cosas. Si algo no carga, `:Lazy`. Si no hay autocompletado, el LSP de ese lenguaje no está (`:Mason`, `:checkhealth vim.lsp`).

Dashboard: solo si abres `nvim` sin archivo. `f` archivo, `g` grep, `r` recientes, `e` árbol, `a` opencode, `m` Mason, `l` Lazy, `q` salir.

Prefijos (Espacio y una letra):

Prefijo	Familia
<code>f</code>	buscar (Telescope)
<code>g</code>	git
<code>b</code>	buffers (y Back si esperas)
<code>w</code>	ventanas
<code>x</code>	errores
<code>m</code>	harpoon
<code>t</code>	terminal
<code>r</code>	ejecutar
<code>d</code>	depurar
<code>fl</code>	Flutter
<code>a</code>	terminales de IA (solo Neovim)
<code>c</code>	Copilot Chat (solo Neovim)

Flujo típico de un archivo: `<leader>ff` lo abres → editas con `ciw` y compañía → `gd` va a la definición si hay LSP → `<leader>fm` formatea → `<leader>gs` git.

11. Plugins de este repo (cuándo, no el catálogo entero)

Tablas largas: [WORKFLOW.md](#). Aquí el *cuándo*.

which-key — Espacio y espera. Es el índice.

telescope — `<leader>ff` archivo, `fg` texto (necesitas ripgrep), `fo` recientes, `fk` atajos. `.,,` también busca archivos.

neo-tree / **oil** — `pv` árbol. `-` o `oe` editar la carpeta como si fuera un buffer (renombrar archivos escribiendo).

harpoon — archivos de cabecera del día: `ma` marca, `1-9` saltan.

Comment / **surround** / **flash** — `gcc` comenta la línea. Surround: `ysi"` pone comillas a la palabra, `ds"` las quita, `cs"` cambia `"` por `'`. Flash: `s` salta a un carácter. **Visual S es surround, no Flash.**

treesitter — colores de verdad. En visual/operador: **af/if** función, **ac/ic** clase (cuando el parser está). Eso es además de **iw/i**".

LSP + cmp + none-ls — **gd** definición, **K** documentación, **gR** referencias, **<leader>rn** renombrar, **ca** code action, **fm** format. Completar: **Ctrl-Space** en insertar. Dart lo arranca flutter-tools (Flutter SDK), no Mason.

trouble — **xx** lista de problemas, **xn/xp** siguiente/anterior error.

git — **gs** estado, **gn** siguiente hunk, **gsa** stage del hunk, **gc** commit, **gp** push. Diff grande: **gdo**. Conflictos de merge: **gco/gct/gcb**.

terminal / runner / tests — **tt** terminal, **jj** para salir del modo terminal. **r** ejecuta. **ten** test más cercano (si el runner del lenguaje existe).

DAP — **db** breakpoint, **dc** continuar, **du** UI. Sin adaptador instalado no hace magia.

Flutter — **fla** run, **flr** reload, **fls** restart, **flq** quit.

Copilot / AI — CopilotChat cc. Visual **as** manda la selección a un CLI (opencode, claude, ...) si está en PATH; si no, te dice cómo instalarlo.

undotree — **<leader>u** en *normal* es el árbol de undo. En *visual*, **u** sigue siendo minúsculas.

12. IdeaVim

Plugins del IDE → IdeaVim → menú **Tools | Vim**. Copia **.ideavimrc** a **~/.ideavimrc**. Recarga con **:source ~/.ideavimrc**.

Los prefijos coinciden. No copies el **.vimrc** clásico dentro de IdeaVim (lleva vim-plug). Atajos de IA y CopilotChat no están en el IDE. Si un **<Action>** no existe: **:actionlist**.

13. Cuando se rompe

Qué ves	Qué mirar
Recuadros en vez de iconos	Fuente Nerd Font y reiniciar el terminal
Plugin en rojo	:Lazy
Completar vacío	:Mason y :checkhealth vim.lsp
Dart mudo	Flutter en PATH; no dupliques dartls
El atajo “no va”	Prefijo de 300 ms; :Telescope keymaps
E37 al salir	hay cambios; :wq o :q!

Orden sugerido en clase

1. Mecanografía + **nvim +Tutor** (una sesión).
2. Modos, **h j k l**, **i/Esc**, **:w :q** (una sesión).
3. Gramática **d/c/y** + movimientos y números (una sesión).
4. Objetos **ciw ci" ci{** y visual (una sesión).
5. Buscar, **:s**, buffers y **Ctrl-w** (una sesión).
6. Registros y una macro tonta (media sesión).
7. Instalar *esta* config y Telescope / git / LSP sobre un archivo real.

Si alguien pide “el atajo de buscar archivo” el día 1, se puede enseñar <leader>ff, pero que sepa que es azúcar. El editor de verdad es el capítulo 4.

Grabación Udemmy (secciones, vídeos, slides): [UDEMY.md](https://www.udemy.com/course/learn-ff/).